TypeORM Transactions

A database transaction symbolizes a unit of work performed within a database management system against a database, and treated in a coherent and reliable way independent of other transactions. A transaction generally represents any change in a database (learn more).

There are many different strategies to handle TypeORM transactions. We recommend using the `QueryRunner` class because it gives full control over the transaction.

First, we need to inject the `DataSource` object into a class in the normal way:

```
@Injectable()
export class UsersService {
  constructor(private dataSource: DataSource) {}
}
```

****

```
import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { User } from './user.entity';
import { DataSource, DeleteResult, Repository } from 'typeorm';

@Injectable()
export class UsersService {

    constructor(
        @InjectRepository(User) private userRepository: Repository<User>,
        private dataSource: DataSource
    ) {}

    async findAll():Promise<User[]> {
        return this.userRepository.find()
    }

    async findOne(id: number) :Promise<User | null> {
        return this.userRepository.findOneBy({ id })
    }

    async remove(id: number): Promise<DeleteResult> {
        return this.userRepository.delete(id)
```

```
    }

    async createMany(users: User[]) {
        const queryRunner = this.dataSource.createQueryRunner()
        await queryRunner.connect()
        await queryRunner.startTransaction()
        try {
            await queryRunner.manager.save(users[0])
            await queryRunner.manager.save(users[1])
            await queryRunner.commitTransaction()
        } catch (e) {
            await queryRunner.rollbackTransaction()
        } finally {
            await queryRunner.release()
        }
    }
}
```

****

Alternatively, you can use the callback-style approach with the `transaction` method of the `DataSource` object (read more).

```
async createMany(users: User[]) {
  await this.dataSource.transaction(async manager => {
    await manager.save(users[0]);
    await manager.save(users[1]);
  });
}
```

****

To use all types of relation in nestjs we must consider the following:

- Avoid the cycle relations between entities when using @JoinColumn
- Avoid unsupported Queries (ex: ILike) when using MariaDB/MySQL
- Use the right configurations for typeorm with DataSource
- The problems when dealing with update one-to-one relationships

****

The DataSource of project:

```
import { DataSource } from "typeorm";

export default new DataSource({
  type: 'mariadb',
  synchronize: false,
  host: "localhost",
  port: 3306,
  username: "root",
  password: "123",
  database: "jloka_nestjs_01",
  entities: [__dirname + '/**/*.entity{.ts,.js}'],
  migrations: [__dirname + '/migrations/*{.ts,.js}'],
})
```

The required packages:

```
"@nestjs/typeorm": "^11.0.0",
"class-transformer": "^0.5.1",
"class-validator": "^0.14.3",
"mysql2": "^3.16.0",
"@nestjs/mapped-types": "^2.1.0",
"typeorm": "^0.3.28" // for cli operations
```

The app module is:

```
@Module({
  imports: [
    ConfigModule.forRoot(),
    TypeOrmModule.forRootAsync({
      imports: [ConfigModule],
      useFactory: (configService: ConfigService) => ({
        type: 'mariadb',
        host: configService.get('DB_HOST'),
        port: configService.get('DB_PORT'),
        username: configService.get('DB_USERNAME'),
        password: configService.get('DB_PASSWORD'),
        database: configService.get('DB_DATABASE'),
        entities: [__dirname + '/**/*.entity{.ts,.js}'],
        migrations: [__dirname + '/migrations/**/*{.ts,.js}'],
        cli: {
          migrationsDir: 'src/migrations',
        },
        synchronize: false,
        logging: true,
      }),
      inject: [ConfigService],
    }),
    UsersModule,
    ProfileModule,
    PostModule,
    CategoryModule
  ],
  controllers: [AppController],
  providers: [AppService],
})
export class AppModule {}
```

The user entity is:

```
import {
  Entity,
  PrimaryGeneratedColumn,
  Column,
  OneToOne,
  OneToMany,
  CreateDateColumn,
  UpdateDateColumn,
```

```ts
} from 'typeorm';
import { Profile } from '../profile/profile.entity';
import { Post } from '../post/post.entity';

@Entity('users')
export class User {
  @PrimaryGeneratedColumn('uuid')
  id: string;

  @Column({ unique: true })
  email: string;

  @Column()
  firstName: string;

  @Column()
  lastName: string;

  @Column({ default: true })
  isActive: boolean;

  // One-to-One with Profile
  @OneToOne(() => Profile, (profile) => profile.user, {
    // cascade: true,
    onDelete: 'CASCADE',
    onUpdate: 'CASCADE'
  })
  // @JoinColumn()
  profile: Profile;

  // One-to-Many with Post
  @OneToMany(() => Post, (post) => post.author)
  posts: Post[];

  @CreateDateColumn()
  createdAt: Date;

  @UpdateDateColumn()
  updatedAt: Date;
}
```

The user service is:

```
import { ConflictException, Injectable, NotFoundException } from
'@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { User } from './user.entity';
import { DataSource, Repository } from 'typeorm';
import { Profile } from 'src/profile/profile.entity';
import { CreateUserDto } from './dto/create-user.dto';
import { UserResponseDto } from './dto/user-response.dto';
import { plainToInstance } from 'class-transformer';
import { UpdateUserDto } from './dto/update-user.dto';

@Injectable()
export class UsersService {

  constructor(
    @InjectRepository(User) private userRepository: Repository<User>,
    @InjectRepository(Profile) private profileRepository: Repository<Profile>,
    private dataSource: DataSource
  ) { }

  async create(createUserDto: CreateUserDto): Promise<UserResponseDto> {
    // Check if user with email already exists
    const existingUser = await this.userRepository.findOne({
      where: { email: createUserDto.email },
    });

    if (existingUser) {
      throw new ConflictException('User with this email already exists');
    }

    return await this.dataSource.transaction(async (transactionalEntityManager)
=> {
      const user = this.userRepository.create(createUserDto);

      // Create profile if provided
      if (createUserDto.profile) {
        const profile = this.profileRepository.create(createUserDto.profile);
        user.profile = profile;
      }

      const savedUser = await transactionalEntityManager.save(user);

      // Load with relations
```

```
      const userWithRelations = await transactionalEntityManager.findOne(User, {
        where: { id: savedUser.id },
        relations: ['profile', 'posts'],
      });

      return plainToInstance(UserResponseDto, userWithRelations, {
        excludeExtraneousValues: true,
      });
    });
  }

  async findAll(): Promise<UserResponseDto[]> {
    const users = await this.userRepository.find({
      relations: ['profile'],
      order: { createdAt: 'DESC' },
    });

    return plainToInstance(UserResponseDto, users, {
      excludeExtraneousValues: true,
    });
  }

  async findOne(id: string): Promise<UserResponseDto> {
    const user = await this.userRepository.findOne({
      where: { id },
      relations: ['profile', 'posts'],
    });

    if (!user) {
      throw new NotFoundException(`User with ID ${id} not found`);
    }

    return plainToInstance(UserResponseDto, user, {
      excludeExtraneousValues: true,
    });
  }

  async update(id: string, updateUserDto: UpdateUserDto):
Promise<UserResponseDto> {
    const user = await this.userRepository.findOne({
      where: { id },
      relations: ['profile'],
    });

    if (!user) {
```

```
      throw new NotFoundException(`User with ID ${id} not found`);
    }


    if (updateUserDto.email && updateUserDto.email !== user.email) {
      const existingUser = await this.userRepository.findOne({
        where: { email: updateUserDto.email },
      });

      if (existingUser) {
        throw new ConflictException('Email already in use');
      }
    }

    // Update user fields
    Object.assign(user, updateUserDto);

    // Update profile if provided
    if (updateUserDto.profile && user.profile) {
      Object.assign(user.profile, updateUserDto.profile);
    }
    // here we must call the profile repository for updating
    const updatedUser = await this.userRepository.save({...user, profile:
user.profile});

    // Reload with relations
    const userWithRelations = await this.userRepository.findOne({
      where: { id: updatedUser.id },
      relations: ['profile', 'posts'],
    });

    return plainToInstance(UserResponseDto, userWithRelations, {
      excludeExtraneousValues: true,
    });
  }

  async remove(id: string): Promise<void> {
    const user = await this.userRepository.findOne({ where: { id } });

    if (!user) {
      throw new NotFoundException(`User with ID ${id} not found`);
    }

    await this.userRepository.remove(user);
  }
```

```
  async searchUsers(searchTerm: string): Promise<UserResponseDto[]> {
    const users = await this.userRepository
      .createQueryBuilder('user')
      .where('LOWER(user.email) LIKE :searchTerm', { searchTerm:
`%${searchTerm.toLowerCase()}%` })
      .orWhere('LOWER(user.firstName) LIKE :searchTerm', { searchTerm:
`%${searchTerm.toLowerCase()}%` })
      .orWhere('LOWER(user.lastName) LIKE :searchTerm', { searchTerm:
`%${searchTerm.toLowerCase()}%` })
      .orderBy('user.createdAt', 'DESC')
      .getMany();

    return plainToInstance(UserResponseDto, users, {
      excludeExtraneousValues: true,
    });
  }


  async createMany(users: User[]) {
    const queryRunner = this.dataSource.createQueryRunner()
    await queryRunner.connect()
    await queryRunner.startTransaction()
    try {
      await queryRunner.manager.save(users[0])
      await queryRunner.manager.save(users[1])
      await queryRunner.commitTransaction()
    } catch (e) {
      await queryRunner.rollbackTransaction()
    } finally {
      await queryRunner.release()
    }
  }
}
```